

Table of contents

List of Figures	2
List of Tables	3
List of Abbreviations	4
1. Overview of internship	1
1.1. Introduction	1
1.2. Artix-7	1
2. Methods	2
2.1. Cooley-Tukey: Radix-2 DIT	2
2.1.1. Implementation in SystemVerilog	4
2.1.2. Schematic	5
2.1.3. Simulation	6
2.1.4. Implementation results	6
2.2. Radix-2 Multi Path Delay Commutator	7
2.2.1. Schematic	8
2.2.2. implementation result	9

List of Figures

Figure 2.1 Steps involved in Cooley-Tukey algorithm	2
Figure 2.2 Signal flow graph of 8-point radix-2 DIT-FFT	3
Figure 2.3 Block diagram of Radix-2 DIT FFT implementation	5
Figure 2.4 Working of FSM in Radix-2 DIT FFT	6
Figure 2.5 Timing diagram of output of 4 point Radix-2 DIT-FFT	6
Figure 2.6 Signal flow graph of 4-point radix-2 DIF-FFT	7
Figure 2.7 Steps in R2MDC algorithm with $N = 4$ case	8
Figure 2.8 Schematic of R2MDC implementation	9

List of Tables

List of Abbreviations

DFT:	Discrete Fourier Transform
DIF:	Decimation in Frequency
DIT:	Decimation in Time
DSP:	Digital Signal Processing
FFT:	Fast Fourier Transform
FT:	Fourier Transform
WNS:	Worst Negative Slack

Chapter 1:

Overview of internship

1.1 Introduction

In Digital Signal Processing (DSP), the need to express a time domain signal to frequency domain often comes in various settings. For this Fourier Transform (FT) is widely employed. However FT is computationally expensive. In past century various algorithms have been proposed to compute FT of signals with limited compute power and memory. Even to this date, various forms of FT algorithms have been proposed. These set are now commonly called Fast Fourier Transform (FFT) algorithms.

My internship is involved in implementing few of these algorithms using SystemVerilog suitable for FPGA. The architecture and operation of these algorithms are detailed in the following chapter, as well as the hardware requirements and some performance aspects are also discussed along with them. In the last chapter, a comparative analysis of all algorithms are outlayed.

For analysis purpose Artix-7 FPGA is used in all the implementation. Operating frequency of 100 MHz is targeted in all algorithms. However as we will some algorithm cannot achieve this while others can go beyond this point.

1.2 Artix-7

Chapter 2:

Methods

2.1 Cooley-Tukey: Radix-2 DIT

Cooley-Tukey algorithm re-expresses the Discrete Fourier Transform (DFT) of an arbitrary composite size $N = N_1 N_2$ in terms of N_1 smaller DFTs of sizes N_2 , recursively, to reduce the computation time to $\mathcal{O}(N \log N)$ for highly composite N . Because of the algorithm's importance, specific variants and implementation styles have become known by their own names, such as Radix-2, Radix-4, mixed-radix, etc. Because the Cooley-Tukey algorithm breaks the DFT into smaller DFTs, it can be combined arbitrarily with any other algorithm for the DFT.

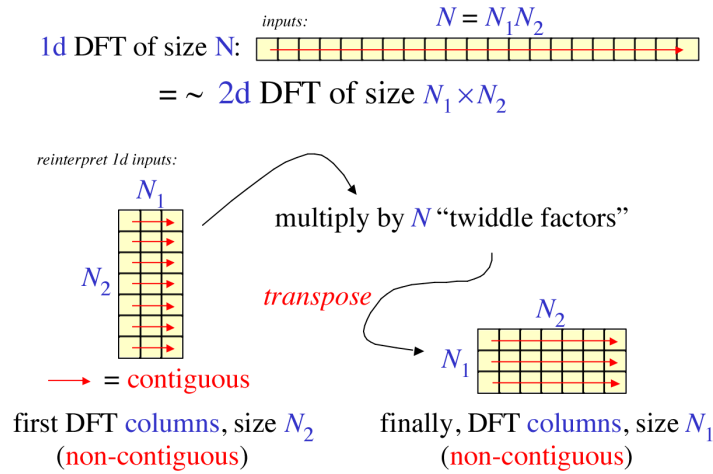


Figure 2.1: Steps involved in Cooley-Tukey algorithm

A radix-2 Decimation in Time (DIT) FFT is the simplest and most common form of the Cooley-Tukey algorithm. Radix-2 DIT divides a DFT of size N into two interleaved DFTs of size $N/2$ with each recursive stage. Hence the overall time complexity of algorithm is $\mathcal{O}(N \log N)$. The following is the derivation of radix-2 DIT:

DFT of sequence x_n is defined by:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-j \frac{2\pi}{N} nk\right)$$

The radix-2 DIT algorithm rearranges the DFT of the function x_n into two parts: a sum over the even-numbered indices $n = 2m$ and a sum over the odd-numbered indices $n = 2m + 1$:

$$\begin{aligned}
 X_k &= \sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{N}(2m)k\right) + \sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{N}(2m+1)k\right) \\
 &= \underbrace{\sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{N}mk\right)}_{\text{DFT of even-indexed part of } x_n} + \exp\left(-j\frac{2\pi}{N}k\right) \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{N}mk\right)}_{\text{DFT of odd-indexed part of } x_n} \\
 &= E_k + \exp\left(-j\frac{2\pi}{N}k\right) O_k
 \end{aligned}$$

Using the periodicity of the complex exponential, $X_{k+\frac{N}{2}}$ is also obtained from E_k and O_k as:

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

We can write X_k and $X_{k+\frac{N}{2}}$ as:

$$X_k = E_k + \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

This result, expressing the DFT of length N recursively in terms of two DFTs of size $N/2$, is the core of the radix-2 DIT FFT. The algorithm gains its speed by re-using the results of intermediate computations to compute multiple DFT outputs. The final outputs are obtained by combination of E_k , O_k and $\exp(-j2\pi k/N)$ (called “twiddle factor” or phase rotation factor) which is simply a size-2 DFT (also called “Butterfly”). The operation can be expressed as a signal flow graph shown in Figure 2.2

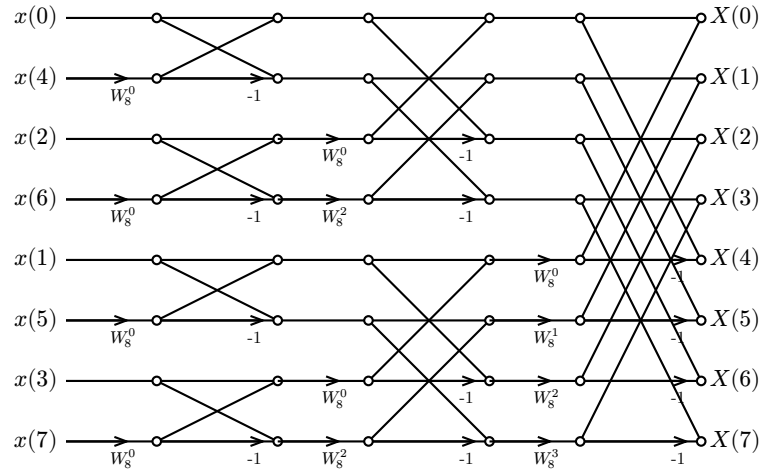


Figure 2.2: Signal flow graph of 8-point radix-2 DIT-FFT

In the software, assuming we have array of input x , the computation begins with reordering the array by swapping element at index i with element at an index which is bit reverse of i . Then subsequently performing radix-2 recursively.

2.1.1 Implementation in SystemVerilog

The SystemVerilog was chosen because of many features which improves ease of development. The implementation accpets a *fixed point* complex number (given through 2 separate buses for real and imaginary part) and returns computed DFT numbers through another 2 separate buses. The operation described below:

1. A reset pulse is applied
2. Start signal is issued which starts a counter inside the module.
3. `input_valid` signal is applied and input is passed in each positive edge of clock.
4. After N clock cycle, when all input are be given, `input_valid` is reset and module starts computing
5. After $N \log_2(N)/2$ clock cycles the output will be available
6. To read output, output read enable is raised and read address is passed. In the next clock cycle the output will be obtained.

The whole process takes $2N + N \log_2(N)/2 + 1$ clock cycles from passing first input to getting last output: $2N$ cycles for applying input and getting output and $N \log_2(N)/2 + 1$ cycles for computation. The module consists of

- 1 butterfly module
- 1 internal memory of size N
- 1 counter
- 1 address generator
- 1 bit reversing module
- 1 twiddle factor generator
- a control logic (FSM)

In theory area wise this should be most area efficient design. The design is also fully parametric – the number of points and data width can be changed. The twiddle factor is generated using a python script. Design is limited in terms of throughput and latency and is not useful for high performance use cases.

2.1.2 Schematic

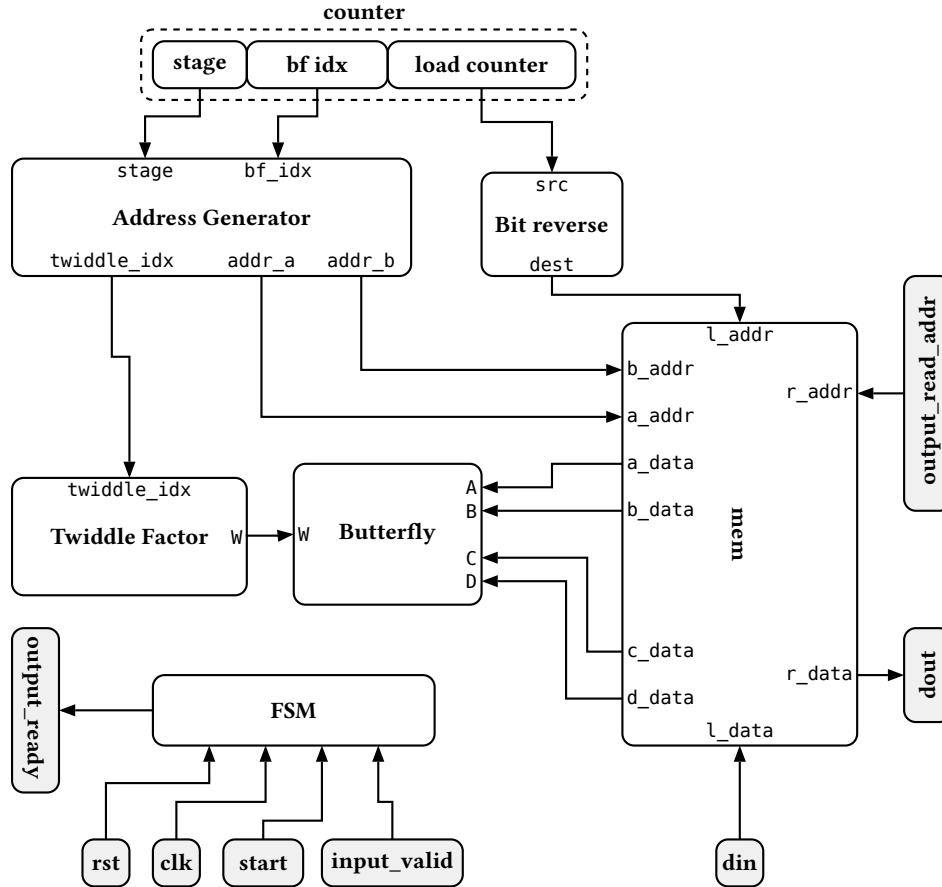


Figure 2.3: Block diagram of Radix-2 DIT FFT implementation

When a start signal is sent, the internal counter starts running. Based on the value of counter address to the addresses of relevant data – inputs to butterfly and twiddle factor – is generated by address generator. A portion of the counter is bit reversed to produce an address that will be used to store the input. The computing modules and FSM all also run in parallel. The FSM has 4 states:

- IDLE (not doing any processing)
- LOAD (loading input to internal memory)
- COMPUTE (computing the results and storing it in the memory)
- DONE: All computations are done and ready for next cycle.

The operation of FSM is summarized in Figure 2.4.

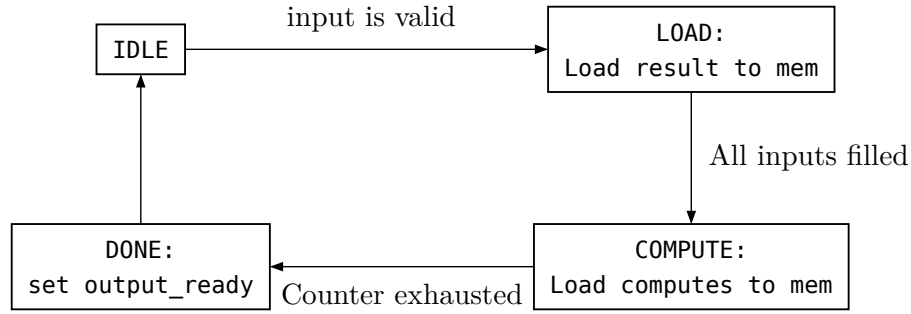


Figure 2.4: Working of FSM in Radix-2 DIT FFT

After the output is ready, the can be read by raising `output_read_en` and passing the address to be read in `output_read_addr`.

2.1.3 Simulation

Here 4 point DFT is computed in which input is real valued ramp signal. In the Figure 2.5, the output is observed at `dout_re` and `dout_im`. The results were verified using *GNU Octave*.

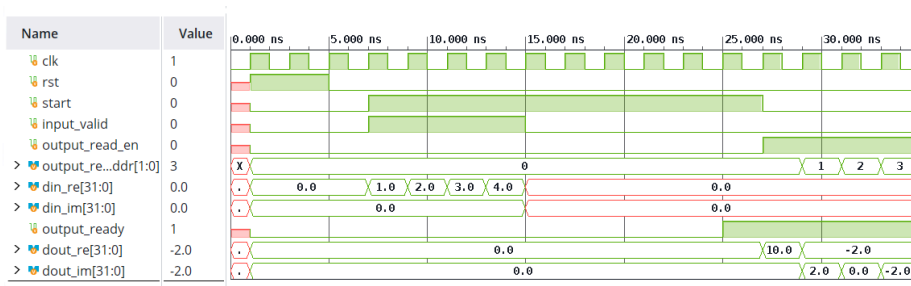


Figure 2.5: Timing diagram of output of 4 point Radix-2 DIT-FFT

2.1.4 Implementation results

The **Artix-7 (part no: xc7a100tcs9324-1)** FPGA was chosen for synthesis. Adjusting for slack, the current design works at maximum of 80.64 MHz. But in vivado the default implementation step still produces a layout which has Worst Negative Slack (WNS) of -0.2ns . Using the TCL command `phys_opt_design -directive Explore` brought it to $+0.095\text{ns}$.

The implementation is by no means efficient for practical use, but serves as a base upon which other algorithms can be implemented and evaluated.

table: for 16 Bit FIXP=10, N=8 obtained by: `report_utilization -hierarchical`

Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs	RAMB36	RAMB18	DSP Blocks
fft_radix2	(top)	1106	1106	0	0	306	0	0	4
(fft_radix2)	(top)	116	116	0	0	306	0	0	0
bs1	butterfly_structure	990	990	0	0	0	0	0	4

report_route_status
Design Route Status

	:	# nets :
-----	:	-----
# of logical nets.....	:	1723 :
# of nets not needing routing.....	:	461 :
# of internally routed nets.....	:	461 :
# of routable nets.....	:	1262 :
# of fully routed nets.....	:	1262 :
# of nets with routing errors.....	:	0 :
-----	:	-----

2.2 Radix-2 Multi Path Delay Commutator

This algorithm is based on the Decimation in Frequency (DIF) method. Structure and flow of this algorithm in the case where $N = 4$ is shown in Figure 2.7. The derivation is as following. Consider a general $N = 4$ DIF Signal flow graph shown in Figure 2.6. To obtain the final output input pairs a , c and b and d must be present at the time of processing. To do this, the first $N/2 = 2$ inputs has to be delayed (using shift registers). So at first, a commutator (shown as a demuxer) commutes initial $N/2$ samples to delay block and commutes rest of the samples to bottom part of butterfly.

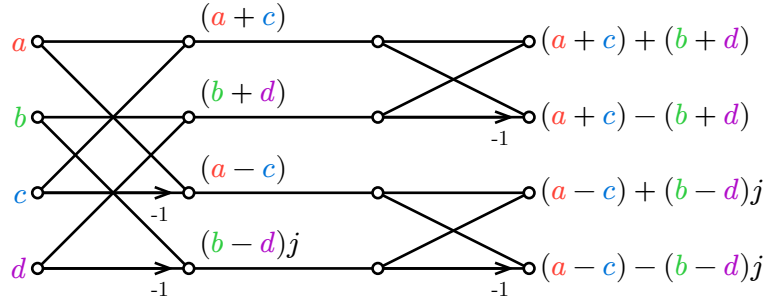
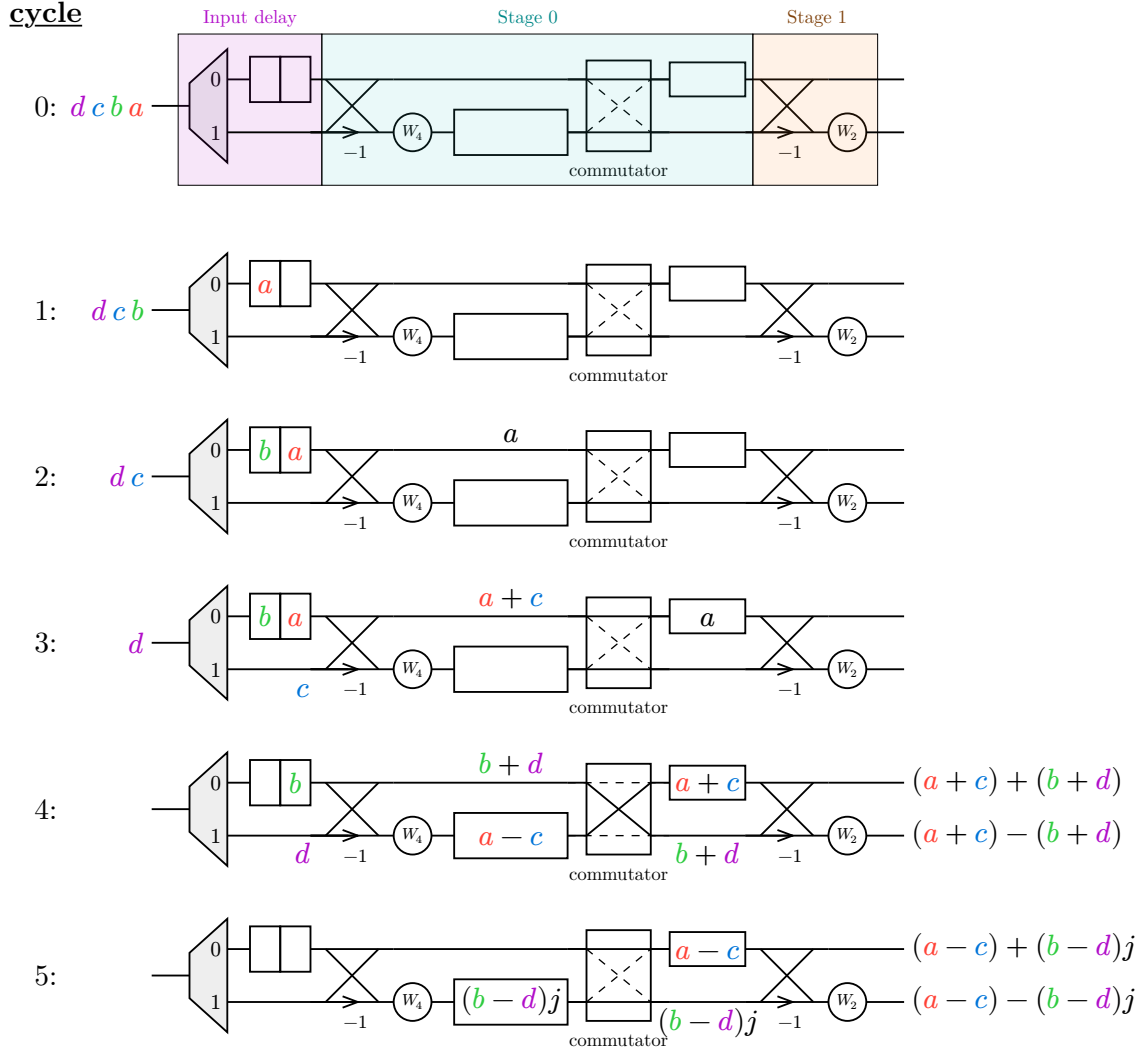


Figure 2.6: Signal flow graph of 4-point radix-2 DIF-FFT

1. In Cycle 0 represents initial state where first input is available at demux.
2. In Cycle 1 and 2, the first 2 inputs are loaded into memory.
3. In Cycle 3, the 3rd input is commuted to bottom part of 1st stage butterfly, the output is available to latch in the flip flops in the next stage.
4. In cycle 4 and 5, all inputs have been given and meaningful outputs from the FFT block will be obtained – $X[0]$ and $X[2]$ in cycle 4 and $X[1]$ and $X[3]$ in cycle 5.


 Figure 2.7: Steps in R2MDC algorithm with $N = 4$ case

2.2.1 Schematic

The design is fully parametric with parameters:

- **STAGES:** number of stages. (hence total number of input points = 2^{STAGES})
- **DW:** Data width (default: 16 bits)
- **FIXP_Q:** Number of bits for decimal part (default: 10)

The design consists of a input delay block that sets the initial delay of $N/2$. The outputs **delayed** and **undelayed** are passed to cascade of R2MDC Stage blocks. Each R2MDC stage is a pipelined design consisting of a butterfly unit, twiddle ROM and commutator. Butterfly here only computes sum and difference of incoming inputs, the multiplication with twiddle factor is done using a pipelined multiplier. The required twiddle factors are computed during synthesis and are stored in a LUT. The control signals are also delayed by the same amount as that of data lines.

In each stage the size of delay element reduces by half. No delays, multipliers and commutators are required in final stage. The final output is obtained at `out0` and `out1` corresponding to bit reversed indexed $X[k]$ and $X[k + N/2]$

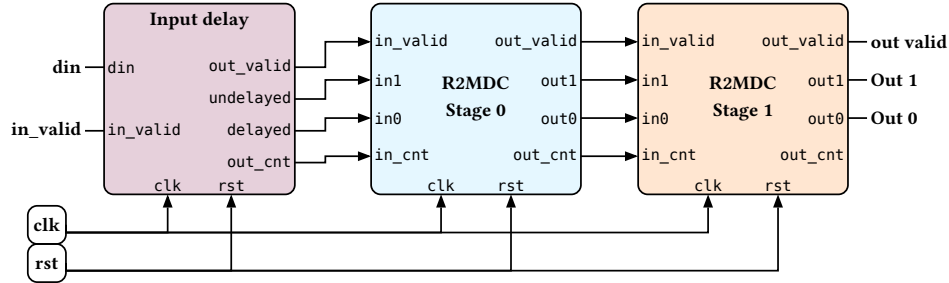


Figure 2.8: Schematic of R2MDC implementation

2.2.2 implementation result

table: for 16 Bit FIXP=10, N=8 obtained by:

```
>> report_utilization -hierarchical
```

Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs	RAMB36	RAMB18	DSP Blocks
R2MDC	(top)	281	248	0	33	404	0	0	8
(R2MDC)	(top)	0	0	0	0	7	0	0	0
gen_stages[1].stage_inst	R2MDC_stage	84	84	0	0	166	0	0	4
(gen_stages[1].stage_inst)	R2MDC_stage	0	0	0	0	68	0	0	0
gen_stage.comm	Commutator	80	80	0	0	98	0	0	0
gen_stages[2].stage_inst	R2MDC_stage__parameterized0	83	82	0	1	99	0	0	4
(gen_stages[2].stage_inst)	R2MDC_stage__parameterized0	1	0	0	1	66	0	0	0
gen_stages[3].stage_inst	R2MDC_stage__parameterized1	0	0	0	0	65	0	0	0
(gen_stages[3].stage_inst)	R2MDC_stage__parameterized1	0	0	0	0	65	0	0	0
s2p	S2P_Buffer	114	82	0	32	67	0	0	0
d_r	Delay_pipe__parameterized3_0	56	40	0	16	32	0	0	0
d_i	Delay_pipe__parameterized3	56	40	0	16	32	0	0	0

```
>> llength [get_nets -hierarchical]
3898
>> report_route_status -show_all
Design Route Status
# nets :
# of logical nets..... : 1541 :
# of nets with no placed pins..... : 1541 :
# of nets not needing routing..... : 0 :
# of routable nets..... : 0 :
# of nets with routing errors..... : 0 :
```